

# 攻击实验室：理解缓冲区溢出漏洞

## 1 介绍

本实验要求针对两个存在不同安全漏洞的程序，共发起五次攻击。通过本次实验，您将掌握以下技能：

- 当程序无法有效防范缓冲区溢出时，您将了解到攻击者可能利用安全漏洞的多种攻击方式。
- 通过本章，您将更好地理解如何编写更安全的程序，以及编译器和操作系统提供的一些功能，使程序不那么容易受到攻击。
- 您将深入理解x86-64机器码的栈和参数传递机制。
- 您将深入了解x86-64指令的编码机制。
- 您将获得使用GDB和objdump等调试工具的更多经验。

**注意：**在本实验中，您将获得利用操作系统和网络服务器中的安全弱点的方法的第一手经验。我们的目的是帮助您了解程序的运行时操作，并了解这些安全弱点的性质，以便您在编写系统代码时可以避免它们。我们不赞成使用任何其他形式的攻击来获得对任何系统资源的未经授权的访问。

您需要参考CS: APP3e手册的第3.10.3和3.10.4节来完成本实验。

## 2 物流

和往常一样，这是一个个人项目。您将为定制生成的目标程序生成攻击。

### 2.1 正在获取文件

您可以通过将Web浏览器指向以下地址来获取文件：

`http://$Attacklab::SERVER_NAME:15513/`

**instructor:** `$Attacklab::server` NAME是运行攻击实验室服务器的机器。您可以在`attacklab/Attacklab.pm`和`attacklab/src/build/driverhdrs.h`中定义它。

服务器将构建您的文件，并将它们以tar文件的形式返回到浏览器中，该tar文件名为`targetk.tar`，其中`k`是目标程序的唯一编号。

**注意：**需要花费数秒时间来构建和下载您的目标，请耐心等待。

将目标文件tar保存到计划使用的受保护Linux目录中，然后执行命令：`tar-xvf targetk.tar`。该操作将解压出一个名为`targetk`的目录，其中包含以下文件。

您只能下载一组文件。如果因某种原因下载了多个目标，请选择一个目标进行处理，然后删除其余目标。

**警告：**如果您在PC上使用Winzip等实用程序或让浏览器进行解压缩来展开`targetk.tar`，那么您将面临重置可执行文件权限位的风险。

`targetk`中的文件包括：

`readme.txt`：一个描述目录内容的文件

`ctarget`：易受攻击的可执行程序*代码注入*攻击

`rtarget`：一个易受*返回导向式程序*攻击的可执行程序

`cookie.txt`：一个8位十六进制代码，您将在攻击中用作唯一标识符。

`farm.c`：目标系统“小工具农场”的源代码，用于生成返回导向型编程攻击。

`hex2raw`：一个用于生成攻击字符串的实用工具。

在以下说明中，我们假设您已将文件复制到受保护的本地目录，并在该目录中执行程序。

## 2.2 重要点

以下是关于本实验有效解决方案的一些重要规则的总结。当你第一次阅读本文档时，这些要点可能不太有意义。它们被列在这里，作为你开始时的规则中心参考。

- 您必须在与生成目标的机器相似的机器上完成作业。
- 您的解决方案可能无法使用攻击来绕过程序中的验证代码。具体来说，您在攻击字符串中为`ret`指令所包含的任何地址都应指向以下目标之一：

- 触摸1、触摸2或触摸3的地址。
  - 您注入的代码的地址
  - 来自设备农场的设备地址。
- 您只能使用文件rtarget中的gadget，且其地址范围需介于函数start\_farm和end\_farm之间。

### 3 目标程序

ctarget和rtarget都从标准输入读取字符串，它们使用下面定义的getbuf函数来实现这一操作：

```
1 unsigned getbuf ( ) 2{
3     字符缓冲区[BUFFER_SIZE];
4     获取 (缓冲区);
5 return 1; 6}
```

函数Gets与标准库函数gets类似，都是从标准输入读取字符串（以 ‘\n’ 或文件结束符终止），并将字符串连同空终止符存储到指定目标位置。在本代码中，目标位置是一个声明为BUFFER\_SIZE字节的数组buf。当目标生成时，BUFFER\_SIZE是特定于您程序版本的编译时常量。

函数Gets ( ) 和gets ( ) 无法判断目标缓冲区是否足够大，以存储读取的字符串。它们只是简单地复制字节序列，可能会超出分配给目标的存储器的界限。

如果用户输入的字符串长度足够短，那么getbuf显然会返回

1、如下执行示例所示：

```
unix> ./ctarget
Cookie: 0x1a7dd803
类型字符串: 请保持简短!
无漏洞。Getbuf返回0x1
正常退货
```

通常，如果输入过长的字符串，就会发生错误：

```
unix> ./ctarget
Cookie: 0x1a7dd803
类型字符串: 这个字符串不是很有趣，但它具有属性...Ouch! : 你造成了分段错误!
下次好运
```

(注意, 这里显示的cookie值与您的不同。) rtarget程序将具有相同的行为。正如错误信息所示, 缓冲区溢出通常会导致程序状态损坏, 从而引发内存访问错误。您的任务是更巧妙地处理ctarget和rtarget输入的字符串, 使它们执行更有趣的操作。这些被称为**漏洞利用字符串**。

ctarget和rtarget都需要多个不同的命令行参数:

-h: 打印可能的命令行参数列表

-q: 不将结果发送到评分服务器

-i FILE: 从文件而不是从标准输入提供输入

您的漏洞字符串通常包含与打印字符的ASCII值不对应的字节值。HEX2RAW程序将使您能够生成这些**原始字符串**。有关如何使用HEX2RAW的更多信息, 请参见附录A。

#### 要点:

- 您的漏洞字符串在任何中间位置都不能包含字节值0x0a, 因为这是ASCII代码换行符 ( '\n' )。当Gets遇到这个字节时, 它会认为你打算终止字符串。
- HEX2RAW需要两个数字的十六进制值, 用一个或多个空格隔开。因此, 如果要创建一个十六进制值为0的字节, 需要将其写成00。要创建word0xdeadbeef, 应将“efbeadede”传递给HEX2RAW (注意小端字节顺序需要反转)。

当您正确完成其中一个关卡后, 您的目标程序将自动向评分服务器发送通知。例如:

```
unix> ./hex2raw < ctarget.l2.txt | ./ctarget
Cookie: 0x1a7dd803
输入字符串: Touch2! : 你调用了touch2 (0x1a7dd803)
针对目标ctarget的二级有效解决方案
已通过: 已将漏洞利用字符串发送至服务器进行验证。
干得好!
```

服务器将测试你的漏洞利用字符串以确保其有效, 并更新Attacklab记分板页面, 显示你的用户ID (根据匿名目标编号列出) 已完成此阶段。

你可以通过将你的Web浏览器指向 `http://$Attacklab: : SERVER_`

`NAME: 15513/scoreboard`来查看记分板

与炸弹实验室不同, 这个实验室中犯错不会受到惩罚。你可以随意向ctarget和rtarget发射任何你喜欢的字符串。

重要提示: 您可以在任何Linux机器上开发解决方案, 但提交时必须使用以下指定机器之一:

| 阶段 | 程序      | 等级 | 方法 | 功能  | 道岔 |
|----|---------|----|----|-----|----|
| 1  | CTARGET | 1  | 西  | 触摸1 | 10 |
| 2  | CTARGET | 2  | 西  | 触控2 | 25 |
| 3  | CTARGET | 3  | 西  | 触控3 | 25 |
| 4  | RTARGET | 2  | 推荐 | 触控2 | 35 |
| 5  | RTARGET | 3  | 推荐 | 触控3 | 5  |

CI: 代码注入

ROP: 返回导向编程

图1: 攻击实验室阶段总结

说明: 请将您在buflab/src/config.c中设置的合法域名列表插入此处。

图1总结了该实验室的五个阶段。可以看到, 前三个阶段涉及对ctarget的代码注入(CI)攻击, 而最后两个阶段涉及对rtarget的返回导向编程(ROP)攻击。

## 4 第1部分: 代码注入攻击

在前三个阶段, 您的攻击字符串将攻击ctarget。该程序的设置方式使得堆栈位置在每次运行时都保持一致, 因此堆栈上的数据可以被当作可执行代码来处理。这些特性使得该程序容易受到攻击, 因为攻击字符串包含可执行代码的字节编码。

### 4.1 1级

在第一阶段, 您不会注入新代码。相反, 您的漏洞利用字符串将重定向程序, 使其执行现有过程。

ctarget内部通过以下C代码的函数测试调用getbuf函数:

```
1 void test () 2{
3 int val;
4 val=getbuf ();
5     printf("No exploit.  Getbuf returned 0x%x\n", val);
6 }
```

当getbuf执行其返回语句(getbuf的第5行)时, 程序通常会在函数test内恢复执行(该函数的第5行)。我们希望改变这种行为。在ctarget文件中, 有一个函数touch1的代码, 其C表示如下:

```
1 void touch1 ()
```

```

2 {
3 vlevel=1;          /*部分验证协议*/
4     printf("Touch1!: You called touch1()\n");
5 验证(1);
6 退出(0); 7}

```

您的任务是让ctarget在getbuf执行其return语句时执行touch1的代码，而不是返回到test。请注意，您的漏洞字符串还可能损坏与此阶段无关的堆栈部分，但这不会造成问题，因为touch1会导致程序直接退出。

#### 一些建议：

- 要设计此级别漏洞利用字符串所需的所有信息都可以通过检查ctarget的反汇编版本来确定。使用objdump-d可获得此反汇编版本。
- 该方案的核心在于：将touch1的起始地址以字节形式存储，当getbuf代码执行完毕时，其末尾的ret指令将自动触发touch1的控制权转移。
- 请务必注意字节顺序。
- 您可能需要使用GDB来逐步执行getbuf的最后几个指令，以确保其执行正确。
- buf在getbuf的栈帧中的放置位置取决于编译时常量BUFFER\_SIZE的值，以及GCC使用的分配策略。您需要检查拆分后的代码，以确定其位置。

## 4.2 2级

第二阶段需要在漏洞利用字符串中注入少量代码。

在ctarget文件中，touch2函数的C语言代码如下所示：

```

1 void touch2(unsigned val)
2 {
3     vlevel = 2;          /* Part of validation protocol */
4     if (val == cookie) {
5         printf("Touch2!: You called touch2(0x%.8x)\n", val);
6         validate(2);
7     } else {
8         printf("Misfire: You called touch2(0x%.8x)\n", val);
9         fail(2);
10    }
11    exit(0);
12 }

```

您的任务是让ctarget执行touch2的代码，而不是返回到test。但是，在此情况下，您必须让touch2显得好像您已经将cookie作为其参数传递了。

#### 一些建议：

- 您需要将注入代码的地址以字节形式定位，确保getbuf代码末尾的ret指令能将控制权转移至该地址。
- 注意，函数的第一个参数是通过寄存器%rdi传递的。
- 您注入的代码应将寄存器设置为cookie地址，随后通过ret指令将控制权转移至touch2的首条指令。
- 请勿在漏洞利用代码中使用jmp或call指令，因为这些指令的目标地址编码难以确定。所有控制权转移都应使用ret指令，即使不执行函数调用时也需如此。
- 有关如何使用工具生成指令序列的字节级表示的讨论，请参见附录B。

### 4.3 第3级

第三阶段还涉及代码注入攻击，但通过传递字符串作为参数。

在ctarget文件中，hexmatch和touch3函数的C语言代码如下所示：

```
1 /* Compare string to hex representation of unsigned value */
2 int hexmatch(unsigned val, char *sval)
3 {
4     char cbuf[110];
5     /* Make position of check string unpredictable */
6     char *s = cbuf + random() % 100;
7     sprintf(s, "%.8x", val);
8     return strncmp(sval, s, 9) == 0;
9 }
10
11 void touch3(char *sval)
12 {
13     vlevel = 3;          /* Part of validation protocol */
14     if (hexmatch(cookie, sval)) {
15         printf("Touch3!: You called touch3(\"%s\")\n", sval);
16         validate(3);
17     } else {
18         printf("Misfire: You called touch3(\"%s\")\n", sval);
19         fail(3);
20     }
21     exit(0);
22 }
```

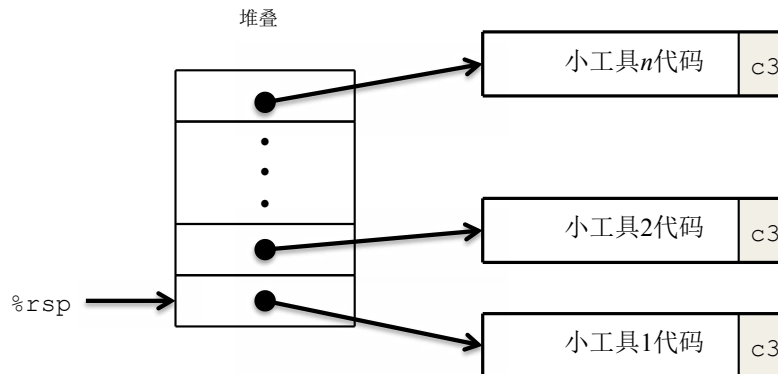


图2：设置执行用的硬件设备序列。字节值0xc3编码ret指令。

您的任务是让ctarget执行touch3的代码，而不是返回到test。您必须让touch3看起来好像您已经将cookie的字符串表示形式作为其参数传递了。

#### 一些建议：

- 您需要在漏洞利用字符串中包含Cookie的十六进制字符串表示。该字符串应由八个十六进制数字组成（按重要性从高到低排序），且不包含前缀“0x”。
- 回想一下，C语言中字符串是用字节序列加上一个0值的字节来表示的。在任何Linux机器上输入“man ascii”，就能看到你需要的字符的字节表示。
- 您注入的代码应将寄存器%rdi设置为该字符串的地址。
- 当调用hexmatch和strncmp函数时，它们会将数据压入堆栈，覆盖getbuf使用的缓冲区内存部分。因此，您需要谨慎处理cookie的字符串表示。

## 5 第二部分：返回导向编程

对目标程序执行代码注入攻击要比对目标程序更困难，因为该程序使用两种技术来阻止此类攻击：

- 它采用随机化技术，使得每次运行时堆栈位置都不相同，从而无法确定注入代码的具体位置。
- 它将存储栈的内存区域标记为不可执行，因此即使您能将程序计数器设置到注入代码的起始位置，程序仍会因分段错误而失败。

幸运的是，聪明的人已经设计出了一些策略，通过执行现有代码而不是注入新代码来在程序中完成有用的事情。这种策略的最通用形式被称为返回导向编程（ROP）[1,2]。ROP策略是识别现有程序中的字节序列



这类指令由一条或多条指令后跟ret指令构成，我们称之为小工具。图2展示了如何通过堆栈配置来执行n个小工具的序列。图中堆栈包含一系列小工具地址，每个小工具由多个指令字节组成，最后一个字节为0xc3，用于编码ret指令。当程序以这种配置执行ret指令时，会触发一系列小工具的执行链路——每个小工具末尾的ret指令都会使程序跳转到下一个小工具的起始位置。

硬件指令库可以利用编译器生成的汇编语言语句代码，特别是函数末尾的指令。虽然这类指令库在实际应用中可能有一定用武之地，但数量有限，难以实现众多关键操作。例如，编译后的函数在返回前几乎不可能以popq%rdi作为最后一条指令。不过好在像x86-64这种以字节为单位的指令集架构，我们可以通过分析指令字节序列中的规律，轻松找到这类指令库。

例如，rtarget的一个版本包含为以下C函数生成的代码：

```
void setval_210 (unsigned *p)
{
    *p = 3347663060U;
}
```

该函数用于攻击系统的可能性似乎微乎其微。但其反汇编代码显示了一个有趣的字节序列：

```
0000000000400f15 <setval_210>:
  400f15:      c7 07 d4 48 89 c7      movl    $0xc78948d4, (%rdi)
  400f1b:      c3                      retq
```

字节序列48 89 c7编码了movq%rax, %rdi指令（具体编码方式可参考图3A）。该序列后紧跟字节c3，用于编码ret指令。该函数起始于地址0x400f15，且序列从函数的第四个字节开始。因此，这段代码包含一个起始地址为0x400f18的代码片段，用于将寄存器%rax中的64位数值复制到寄存器%rdi。

rtarget的代码中包含许多与上文所示setval\_210函数相似的函数，这些函数位于我们称之为gadgets农场的区域中。您的任务是识别gadgets农场中的有用gadgets，并使用这些gadgets执行类似于第2阶段和第3阶段的攻击。

**重要提示：**在rtarget副本中，gadget farm由start\_farm和end\_farm函数界定。请勿尝试从程序代码的其他部分构建gadget。

## 5.1 2级

在第4阶段，您将重复第2阶段的攻击，但使用的是RTARGET，并使用您的gadget农场中的gadget。您可以使用以下指令类型的gadget来构建解决方案，并且只使用前8个x86-64寄存器（%rax-%rdi）。

### A. movq指令的编码

movq  $S, D$

| Source<br>$S$ | Destination $D$ |          |          |          |          |          |          |          |
|---------------|-----------------|----------|----------|----------|----------|----------|----------|----------|
|               | %rax            | %rcx     | %rdx     | %rbx     | %rsp     | %rbp     | %rsi     | %rdi     |
| %rax          | 48 89 c0        | 48 89 c1 | 48 89 c2 | 48 89 c3 | 48 89 c4 | 48 89 c5 | 48 89 c6 | 48 89 c7 |
| %rcx          | 48 89 c8        | 48 89 c9 | 48 89 ca | 48 89 cb | 48 89 cc | 48 89 cd | 48 89 ce | 48 89 cf |
| %rdx          | 48 89 d0        | 48 89 d1 | 48 89 d2 | 48 89 d3 | 48 89 d4 | 48 89 d5 | 48 89 d6 | 48 89 d7 |
| %rbx          | 48 89 d8        | 48 89 d9 | 48 89 da | 48 89 db | 48 89 dc | 48 89 dd | 48 89 de | 48 89 df |
| %rsp          | 48 89 e0        | 48 89 e1 | 48 89 e2 | 48 89 e3 | 48 89 e4 | 48 89 e5 | 48 89 e6 | 48 89 e7 |
| %rbp          | 48 89 e8        | 48 89 e9 | 48 89 ea | 48 89 eb | 48 89 ec | 48 89 ed | 48 89 ee | 48 89 ef |
| %rsi          | 48 89 f0        | 48 89 f1 | 48 89 f2 | 48 89 f3 | 48 89 f4 | 48 89 f5 | 48 89 f6 | 48 89 f7 |
| %rdi          | 48 89 f8        | 48 89 f9 | 48 89 fa | 48 89 fb | 48 89 fc | 48 89 fd | 48 89 fe | 48 89 ff |

### B. Encodings of popq instructions

| Operation | Register $R$ |      |      |      |      |      |      |      |
|-----------|--------------|------|------|------|------|------|------|------|
|           | %rax         | %rcx | %rdx | %rbx | %rsp | %rbp | %rsi | %rdi |
| popq $R$  | 58           | 59   | 5a   | 5b   | 5c   | 5d   | 5e   | 5f   |

### C. Encodings of movl instructions

movl  $S, D$

| Source<br>$S$ | Destination $D$ |       |       |       |       |       |       |       |
|---------------|-----------------|-------|-------|-------|-------|-------|-------|-------|
|               | %eax            | %ecx  | %edx  | %ebx  | %esp  | %ebp  | %esi  | %edi  |
| %eax          | 89 c0           | 89 c1 | 89 c2 | 89 c3 | 89 c4 | 89 c5 | 89 c6 | 89 c7 |
| %ecx          | 89 c8           | 89 c9 | 89 ca | 89 cb | 89 cc | 89 cd | 89 ce | 89 cf |
| %edx          | 89 d0           | 89 d1 | 89 d2 | 89 d3 | 89 d4 | 89 d5 | 89 d6 | 89 d7 |
| %ebx          | 89 d8           | 89 d9 | 89 da | 89 db | 89 dc | 89 dd | 89 de | 89 df |
| %esp          | 89 e0           | 89 e1 | 89 e2 | 89 e3 | 89 e4 | 89 e5 | 89 e6 | 89 e7 |
| %ebp          | 89 e8           | 89 e9 | 89 ea | 89 eb | 89 ec | 89 ed | 89 ee | 89 ef |
| %esi          | 89 f0           | 89 f1 | 89 f2 | 89 f3 | 89 f4 | 89 f5 | 89 f6 | 89 f7 |
| %edi          | 89 f8           | 89 f9 | 89 fa | 89 fb | 89 fc | 89 fd | 89 fe | 89 ff |

### D. Encodings of 2-byte functional nop instructions

| Operation |        | Register $R$ |       |       |       |
|-----------|--------|--------------|-------|-------|-------|
|           |        | %al          | %cl   | %dl   | %bl   |
| andb      | $R, R$ | 20 c0        | 20 c9 | 20 d2 | 20 db |
| orb       | $R, R$ | 08 c0        | 08 c9 | 08 d2 | 08 db |
| cmpb      | $R, R$ | 38 c0        | 38 c9 | 38 d2 | 38 db |
| testb     | $R, R$ | 84 c0        | 84 c9 | 84 d2 | 84 db |

图3：指令的字节编码。所有数值均以十六进制表示。

`movq` : The codes for these are shown in Figure 3A.

`popq` : The codes for these are shown in Figure 3B.

`ret` : This instruction is encoded by the single byte `0xc3`.

`nop` : This instruction (pronounced “no op,” which is short for “no operation”) is encoded by the single byte `0x90`. Its only effect is to cause the program counter to be incremented by 1.

- 所有需要的工具都可以在由`start_farm`和`mid_farm`函数划定的`rtarget`代码区域中找到。
- 你只需要两个小工具就能完成这个攻击。
- 当某个硬件漏洞利用程序使用`popq`指令时，会从栈中弹出数据。因此，你的漏洞利用字符串将包含硬件漏洞地址和数据的组合。

## 5.2 第3级

在开始第五阶段之前，请先停下来考虑一下您到目前为止所取得的成就。在第二和第三阶段中，您使一个程序执行您自己设计的机器代码。如果`ctarget`是一个网络服务器，您就可以向远程机器注入您自己的代码。在第四阶段中，您绕过了现代系统用来阻止缓冲区溢出攻击的两个主要设备。虽然你没有编写自己的代码，但成功注入了一种通过拼接现有代码序列来运行的程序。你在实验中获得95/100的高分。如果你有其他紧急事务，建议立即停止。

第五阶段要求你对目标地址发起ROP攻击，通过指向`cookie`字符串的指针调用`touch3`函数。这看似与使用ROP攻击调用`touch2`的难度相差无几，但关键在于我们刻意设置了这种差异。此外，第五阶段仅计5分，这并不能真实反映所需付出的努力。不妨将其视为课程的额外挑战题，适合那些希望突破常规学习目标的同学。

为了解决第五阶段，您可以在由`start_farm`和`end_farm`函数划定的`rtarget`代码区域中使用`gadget`。除了第四阶段使用的`gadget`外，这个扩展的`farm`还包括不同`movl`指令的编码，如图3C所示。该`farm`部分的字节序列还包含作为功能`nops`的2字节指令，即。这些指令不会改变寄存器或内存中的任何值。如图3D所示，例如`andb%al, %al`这类指令，虽然作用于寄存器的低位字节，但不会改变其数值。

### 一些建议：

- 您需要查看`movl`指令对寄存器高4字节的影响，具体说明可参见教材第183页。
- 官方解决方案需要8个设备（并非全部都是独特的）。

祝你好运，玩得开心！

## A 使用HEX2 RAW

HEX2RAW的输入是一个十六进制格式的字符串。在这种格式中，每个字节值由两个十六进制数字表示。例如，字符串“012345”可以输入为十六进制格式“30 31 32 33 34 35 00”。（请记住，十进制数字 $x$ 的ASCII码是 $0x3x$ ，字符串的结尾由空字节表示。）

传递给HEX 2RAW的十六进制字符之间应有空白（空格或换行符）分隔。建议在编写漏洞利用字符串时，用换行符分隔不同的部分。HEX 2RAW支持C样式的块注释，因此可以标记漏洞利用字符串的不同部分。例如：

```
48 c7 c1 f0 11 40 00 /* mov      $0x40011f0,%rcx */
```

请确保在起始和结束注释字符串（“/\*”、“\*/”）周围留出空间，以便正确忽略这些注释。

如果在文件exploit.txt中生成了十六进制格式的漏洞利用字符串，那么可以使用多种不同的方法将原始字符串应用到ctarget或rtarget：

1. 您可以设置一系列管道，将字符串通过HEX 2RAW。

```
unix> cat exploit.txt | ./hex2raw | ./ctarget
```

2. 您可以将原始字符串存储在文件中，并使用I/O重定向：

```
unix> ./hex2raw < exploit.txt > exploit-raw.txt
unix> ./ctarget < exploit-raw.txt
```

此方法也可用于从GDB内运行时：

```
unix> gdb ctarget
(gdb) 运行<exploit-raw.txt
```

3. 您可以将原始字符串存储在文件中，并将该文件名作为命令行参数提供：

```
unix> ./hex2raw < exploit.txt > exploit-raw.txt
unix> ./ctarget -i exploit-raw.txt
```

此方法也可用于从GDB内运行时。

## B Generating Byte Codes

使用GCC作为汇编器和objdump作为反汇编器，可以方便地生成指令序列的字节代码。例如，假设您编写了一个文件example.s，其中包含以下汇编代码：

#手动生成的汇编代码示例

```
pushq    $0xabcdef        #将值压入栈

addq     $17,%rax          #将17加到%rax

movl     %eax,%edx         #将低32位复制到%edx
```

代码可以包含指令和数据的混合。任何位于 '#' 字符右边的内容都是注释。  
现在可以组装和拆卸此文件：

```
unix> gcc -c example.s
```

```
unix>objdump-d example.o>example.d 生成的文件
```

example.d包含以下内容：

示例.o: 文件格式elf64-x86-64

拆卸.txt部分：

```
0000000000000000 <.text>:
```

```
0: 68 ef cd ab 00          pushq   $0xabcdef

5: 48 83 c0 11             将$0x11加到%rax

9: 89 c2                  mov     %eax,%edx
```

底部的代码行展示了汇编语言指令生成的机器码。每行左侧的十六进制数字表示指令的起始地址（以0开头），而冒号后的十六进制数字则对应指令的字节码。例如，指令push\$0xABCDEF对应的十六进制字节码为68 ef cd ab 00。

通过该文件，您可以获取代码的字节序列：

```
68 ef cd ab 00 48 83 c0 11 89 c2
```

然后将此字符串传递给HEX 2RAW生成目标程序的输入字符串。或者，您也可以编辑example.d以省略多余值，并包含C样式注释以提高可读性，结果如下：

```
68 ef cd 水 00 /* pushq $0xabcdef */
48 83 c0 11 /*添加 $0x11,%rax */
89 c2 /* mov %eax,%edx */
```

这也是一个有效的输入，可以在发送到目标程序之前通过HEX 2RAW。

## 参考文献

- [1] R.Roemer, E.Buchanan, H.Shacham, 和S.Savage.以返回为导向的编程：系统、语言和应用。  
*ACM Transactions on Information System Security*, 15(1): 2: 1–2: 34,2012年3月。
- [2] E.J.Schwartz, T.Avgerinos, 和 D.Brumley.Q: Exploit hardening made easy.在 *USENIX Security Symposium*, 2011.